PREDICTIVE ANALYSIS
PROJECT REPORT
(Project Semester July-November 2026)

BANGALORE TRAFFIC PREDICTION USING MACHINE LEARNING

Submitted by

Riya Thakur
Registration No. 12418725
Programme and Section: B.Tech, K3P24FV
Course Code: INT234

Under the Guidance of

Dr. Madhu Bala

Discipline of CSE/IT
Lovely School of Computer Science and Engineering
Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that **Riya Thakur** bearing Registration no. **12418725** has completed **INT234** project titled, "**Bangalore Traffic Prediction using Machine Learning**" under my guidance and supervision. To the best of my knowledge, the present work is the result of her original development, effort and study.

Signature and Name of the Supervisor: Dr. Madhu Bala

School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab.

Date: 31 October 2026

DECLARATION

I, **Riya Thakur**, student of **B.Tech** under CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 31 October 2026

Registration No. 12418725

Signature

Riya Thakur

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty coordinator, **Dr. Madhu Bala**, for her guidance, encouragement and valuable suggestions during the course of this project. Her explanation of the concepts of predictive analytics helped me to plan the work in a systematic way and to look at the results critically.

I am thankful to Lovely Professional University and the School of Computer Science and Engineering for providing the learning environment and the resources that made this project possible. I also thank TomTom for providing a free developer platform and the Traffic Flow API, which was the source of the real-time data used in this work, and the developers of the open-source Python libraries pandas, NumPy, scikit-learn, matplotlib and seaborn.

Finally, I thank my family and friends for their constant support and motivation, which helped me to complete the project within the time available.

Riya Thakur

TABLE OF CONTENTS

1. Introduction **1**
1.1 Background and motivation 1
1.2 Problem statement 1
1.3 Objectives 1
1.4 Scope of the project 2
1.5 Tools and technologies 2
1.6 Organisation of the report 3
1.7 Overall workflow of the project 3
1.8 Predictive analytics and machine learning concepts 4
2. Source of Dataset **6**
2.1 The TomTom Traffic Flow API 6
2.2 Data collection methodology 6
2.3 Monitored junctions 7
2.4 Description of the dataset 8
2.5 Summary of the collected data 8
2.6 Limitations of the data source 9
3. EDA Process **10**
3.1 Loading and inspecting the data 10
3.2 Data cleaning 10
3.3 Feature engineering 11
3.4 Descriptive statistics 12
3.5 Distribution of the congestion ratio 12
3.6 Current speed versus free-flow speed 13
3.7 Correlation analysis 14
3.8 Defining congestion levels 15
3.9 Coverage of the data over time 16
3.10 Data preprocessing for modelling 16
4. Analysis on Dataset **18**
4.1 Analysis 1: Junction-wise congestion analysis 18
4.2 Analysis 2: Time-of-day analysis 21
4.3 Analysis 3: Regression models (simple, multiple and polynomial) 24
4.4 Analysis 4: Classification of the congestion level 28
4.5 Analysis 5: Clustering of junctions 33
4.6 Analysis 6: Association rules and market basket analysis 38
4.7 Analysis 7: Dimensionality reduction with PCA 40
4.8 Analysis 8: Feedforward neural network (MLP) 43

4.9 Analysis 9: Ensemble learning, Random Forest and the bias-variance trade-off.....45
4.10 Overall discussion and reliability of the results.....50
5. Conclusion.....**51**
6. Future Scope.....**53**
7. References.....**54**

1. Introduction

1.1 Background and motivation

Bangalore, the capital of Karnataka, is one of the fastest growing metropolitan regions in India and a global hub for information technology, start-ups and research. This growth has been accompanied by a very rapid increase in the number of registered vehicles, while the road network has not expanded at the same pace. As a result, long queues at major junctions, slow average speeds and unpredictable travel times have become a part of everyday life for commuters, delivery services, taxi operators and emergency responders. Congestion wastes time and fuel, increases vehicular emissions and adds to the stress of travellers.

Traditional approaches to understanding traffic rely on manual surveys, fixed road-side sensors or occasional reports from traffic police. These methods are expensive, cover only a few locations and are slow to update. In recent years, navigation platforms have started to estimate traffic conditions continuously by aggregating anonymous speed information from connected vehicles, mobile devices and road sensors. Such data is exposed through web Application Programming Interfaces (APIs), which makes it possible for students and researchers to collect real-time observations of road conditions without owning any hardware.

This project uses one such source, the TomTom Traffic Flow API, to collect real-time observations for 24 important locations of Bangalore, to explore the collected data and to build machine learning models that estimate the level of congestion from the time of day, the day of the week and the characteristics of the location. The work combines the stages that are expected in a predictive analytics project: data acquisition, cleaning, exploratory data analysis (EDA), feature engineering, model building, evaluation and interpretation.

1.2 Problem statement

Given the location of a junction in Bangalore and the time at which a vehicle plans to travel through it, the aim is to estimate how congested the road will be. Congestion is expressed in two ways: as a continuous number called the congestion ratio (current speed divided by free-flow speed), and as one of three categories, Low, Medium and High. The problem is therefore treated as a regression task for the continuous value and as a classification task for the category.

1.3 Objectives

The objectives of the project are as follows.

2. Source of Dataset

2.1 The TomTom Traffic Flow API

TomTom is a location technology company that provides maps, navigation and traffic information. Its developer platform offers a Traffic Flow Segment Data service which, for a given latitude and longitude, returns the traffic condition of the road segment that is closest to that point. The service is accessed with a personal API key that is obtained by registering on the TomTom developer portal. A free tier is available for evaluation and learning, with a daily limit on the number of requests, and this limit decided how many junctions could be sampled and how frequently.

For every request the API returns, among other fields, the current average speed on the segment, the free-flow speed (the speed that a driver would be expected to reach in light traffic), the current travel time, the free-flow travel time, a confidence value between 0 and 1 that indicates the quality of the estimate, and the coordinates that describe the segment geometry. These values are estimates produced by TomTom from probe data and are not direct measurements made by the author.

2.2 Data collection methodology

A Python script was written to collect the data automatically. The script holds a dictionary of 24 locations, each with a name and a latitude-longitude pair. It repeats the following steps continuously: for each location it sends one HTTPS request to the Flow Segment Data endpoint, extracts the required fields from the JSON response, adds the current date-time, the hour of the day and the day of the week, and appends one row to a CSV file. After all 24 locations have been processed the script saves the file and waits for 15 minutes before the next round.

Some requests failed from time to time because of network problems or certificate errors on the Wi-Fi connection. To make the collection robust, a later version of the script retries every failed request up to three times with a short pause before giving up on that reading. The script opens the CSV file in append mode, so that when the collection was stopped, for example when the laptop was taken to college, and started again later, new rows were added below the existing rows and no earlier data was lost. The API key was kept inside the script only and is not shared or published.

2.3 Monitored junctions

The 24 locations were chosen to represent the main commercial areas, the Outer Ring Road corridor, the northern and eastern approaches to the city and several residential neighbourhoods. The coordinates are approximate points near each junction. The API snaps each point to the nearest road segment, and the segment that is returned may differ in length and road class from one location to another. The locations and the coordinates that were supplied to the API are listed in Table 2.

Table 2: Monitored junctions and the coordinates supplied to the API

No.	Junction	Latitude	Longitude
1	Silk Board	12.9177	77.6233
2	Marathahalli	12.9569	77.7011
3	Hebbal	13.0358	77.5970
4	KR Puram	13.0012	77.6960
5	Electronic City	12.8452	77.6602
6	Indiranagar	12.9784	77.6408
7	MG Road	12.9756	77.6068
8	Whitefield	12.9698	77.7500
9	Koramangala	12.9352	77.6245
10	Majestic	12.9767	77.5713
11	Yeshwanthpur	13.0285	77.5409
12	Bellandur	12.9304	77.6784
13	Jayanagar	12.9250	77.5938
14	BTM Layout	12.9166	77.6101
15	HSR Layout	12.9116	77.6474
16	Banashankari	12.9255	77.5468
17	Rajajinagar	12.9916	77.5520
18	Malleshwaram	13.0035	77.5646
19	Domlur	12.9609	77.6387
20	Hennur	13.0358	77.6400
21	Mekhri Circle	13.0140	77.5800
22	Nagawara	13.0450	77.6200
23	Kengeri	12.9170	77.4830
24	Richmond Circle	12.9650	77.5970

2.4 Description of the dataset

The collected dataset has 11 attributes. Five of them describe the time and place of the reading, and the remaining six come directly from the API response. The attributes, their data types and their meaning are given in Table 3.

Table 3: Attributes of the collected dataset

Attribute	Type	Description
timestamp	Date-time	Date and time at which the reading was recorded
hour	Integer (0-23)	Hour of the day extracted from the timestamp
weekday	Integer (0-6)	Day of the week, 0 = Monday and 6 = Sunday
location	Text	Name of the junction
lat, lon	Decimal	Coordinates supplied to the API
current_speed	Integer (km/h)	Current average speed on the road segment
free_flow_speed	Integer (km/h)	Speed expected on the segment in light traffic
current_travel_time	Integer (s)	Time needed to travel the segment at the current speed
free_flow_travel_time	Integer (s)	Time needed to travel the segment at free-flow speed
confidence	Decimal (0-1)	Reliability of the estimate given by the API

2.5 Summary of the collected data

The final dataset contains 1418 readings for 24 junctions. The first reading was recorded on 29 October 2026 at 19:24 and the last reading used in this report was recorded on 31 October 2026 at 08:51. Every junction has between 57 and 61 readings. The small difference in the number of readings per junction arises because the last 12 junctions were added to the script slightly later than the first 12, and because a few requests could not be completed even after several attempts. Figure 2 shows the number of readings for each junction and Table 4 summarises the collection by day.

Table 4: Summary of data collection by day

Date	Day	Readings	Mean congestion ratio
2026-10-29	Thursday	418	0.855
2026-10-30	Friday	688	0.892
2026-10-31	Saturday	312	0.985

3. EDA Process

3.1 Loading and inspecting the data

The CSV file was loaded into a pandas DataFrame with the timestamp column parsed as a date-time. The first inspection consisted of checking the number of rows and columns, the data type of every column, the number of missing values and the number of duplicate rows. The dataset has 1418 rows and 11 columns. All numeric columns were read as integers or decimals and the timestamp column was read as date-time, so no type conversions were necessary.

3.2 Data cleaning

Data cleaning was carried out in the following steps.

- Missing values: the number of missing values was zero in every column, so no imputation was required.
- Duplicates: no exact duplicate rows were found. Two readings of the same junction may look similar when they are taken a minute apart, but they have different timestamps and are valid separate observations.
- Invalid free-flow speed: rows with a free-flow speed of zero or less would make the congestion ratio undefined. No such rows were found, but the filter is kept in the pipeline for safety.
- Confidence: the lowest confidence value is 0.766 and only four readings are below 0.8. These readings were retained because the confidence is still high and removing them would not change the results in a meaningful way.
- Upper limit of the ratio: in none of the readings was the current speed higher than the free-flow speed, so the ratio never exceeded 1.0. As a safeguard, the pipeline still limits the ratio to a maximum of 1.0 so that the value always lies between 0 and 1.

The outcome of these checks is summarised in Table 5. After the checks, the cleaned dataset contained all 1418 rows.

Table 5: Summary of data quality checks

Check	Result	Action taken
Missing values	0 in all columns	None required
Duplicate rows	0	None required
Free-flow speed of zero or less	0 rows	Filter kept in the pipeline as a safeguard

3.4 Descriptive statistics

The descriptive statistics of the main numeric attributes are given in Table 7. The average current speed over all readings is 28.6 km/h, compared with an average free-flow speed of 31.7 km/h. The mean congestion ratio is 0.901, which means that, on average, the observed speed is about 10 per cent below the free-flow speed. The median ratio is exactly 1.0, which shows that for more than half of the readings the roads were running at free-flow speed. These readings are mostly from the night hours that dominate the dataset.

Table 7: Descriptive statistics of the dataset (1418 readings)

Attribute	Mean	Std. dev.	Min	Median	Max
Current speed (km/h)	28.6	8.5	11	27	56
Free-flow speed (km/h)	31.7	7.9	21	31	56
Current travel time (s)	924.0	660.3	49	747	2749
Free-flow travel time (s)	814.4	572.3	49	697	1933
Delay (s)	109.6	182.1	0	0	1248
Congestion ratio	0.901	0.132	0.424	1.000	1.000
Confidence	0.998	0.015	0.766	1.000	1.000

The travel time values have a very wide range, from 49 seconds to 2,749 seconds. This is not caused by traffic alone. It is caused by the different lengths of the road segments that the API returned for different junctions. For this reason, comparisons between junctions in this report use the congestion ratio and the delay percentage and not the raw travel time.

3.5 Distribution of the congestion ratio

Figure 3 shows the histogram of the congestion ratio. The distribution is strongly skewed to the right, with a large peak at exactly 1.0. This peak represents free-flowing traffic and it is large because many readings were taken late at night. The remaining readings spread over values from about 0.42 to 0.99, which is the range in which congestion is observed. The lowest ratio recorded is 0.424, at a junction where the current speed was less than half of the speed expected in free flow.

3.7 Correlation analysis

The correlation matrix of the numeric attributes is shown in Figure 5, and the correlation of the congestion ratio with the other attributes is listed in Table 8. The strongest relationship is with the delay, which is expected because both are derived from the same quantities. Among the original attributes, the hour of the day is negatively correlated with the ratio (-0.44). This agrees with the observation that the late-evening hours have more congestion than the night hours, which have smaller hour numbers. The correlation with the day of the week is positive (0.34), but this should not be read as a real weekday effect. In this dataset, the readings of Saturday were taken only in the night and early morning when the roads are empty, so the day number is mixed up with the hour coverage.

Table 8: Correlation of the congestion ratio with other attributes

Attribute	Correlation with congestion ratio
Delay	-0.77
Current speed	0.52
Hour of day	-0.44
Day of week	0.34
Current travel time	-0.21
Free-flow speed	0.03
Free-flow travel time	0.00

The free-flow speed and the free-flow travel time have almost no correlation with the ratio. This confirms that the ratio is a fair measure that does not favour fast or slow roads.

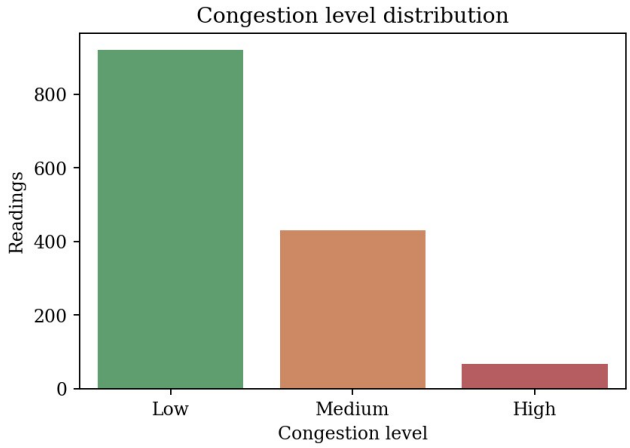


Figure 6: Number of readings in each congestion level

3.9 Coverage of the data over time

Figure 7 shows the hourly average of the congestion ratio over the collection period. The gaps in the line correspond to the hours for which no readings were collected. The figure shows clearly the pattern that is analysed further in Chapter 4: the ratio falls in the evening of Thursday and Friday, rises to almost 1.0 after midnight and falls again in the morning and at midday of Friday.

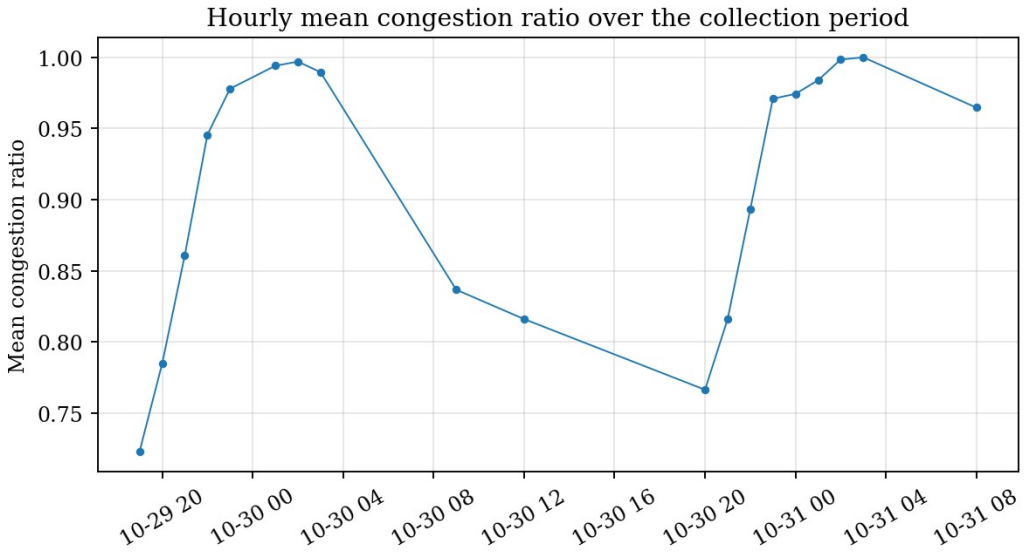


Figure 7: Hourly mean congestion ratio over the collection period

3.10 Data preprocessing for modelling

Data preprocessing is the set of steps that convert the cleaned data into a form that the algorithms can use. The following steps were applied before the models of Chapter 4 were trained.

4. Analysis on Dataset

Nine analyses were carried out on the cleaned dataset, and they cover the main topics of the Predictive Analysis course. The first two are descriptive and answer the questions where and when congestion occurs. Analyses 3 and 4 are supervised learning: regression (simple, multiple and polynomial regression estimated by OLS) and classification (logistic regression, decision tree, support vector machine and ensemble models). Analyses 5 to 7 are unsupervised learning and pattern detection: K-Means and hierarchical clustering, association rules and market basket analysis, and principal component analysis. Analysis 8 uses a feedforward neural network (multi-layer perceptron), and Analysis 9 studies ensemble learning, random forests and the bias-variance trade-off. Each analysis is described under the headings introduction, general description, specific requirements, functions and formulas, analysis results and visualization.

4.1 Analysis 1: Junction-wise congestion analysis

i. Introduction

The purpose of this analysis is to find out which of the 24 monitored junctions were the most congested and which were the least congested during the collection period. This information is useful for commuters who want to avoid heavily loaded roads, and for planners who need to decide where to focus their efforts.

ii. General description

For each junction all readings were grouped together and the mean of the congestion ratio, the minimum ratio, the mean speed and the mean delay percentage were calculated. The junctions were then ranked by the mean congestion ratio, where a lower mean ratio means that vehicles were on average slower than free-flow speed by a bigger margin. The delay percentage was added as a second measure because it expresses the extra time as a share of the free-flow travel time.

iii. Specific requirements, functions and formulas

The analysis needs the cleaned dataset with the derived features congestion_ratio and delay_pct. In pandas, the grouping was done with the groupby function and the statistics were calculated with the agg function. The results were sorted with sort_values and plotted as horizontal bar charts using the barplot function of seaborn. The measures are defined as follows.

Mean Congestion Ratio (junction) = (1/n) x Sum of Ratio over all readings of the junction

Delay % = (Current Travel Time - Free-Flow Travel Time) / Free-Flow Travel Time x 100

iv. Analysis results

The ranking of all junctions is given in Table 10. The most congested junction during the observation period was Indiranagar, with a mean ratio of 0.834 and an average delay of 26.8 per cent over the free-flow travel time. It was followed closely by Hebbal (0.839), Bellandur (0.842), MG Road (0.855) and Marathahalli (0.857). The least congested locations were Electronic City (0.994), BTM Layout (0.978) and Jayanagar (0.956).

Table 10: Junction-wise congestion results (ranked from most to least congested)

Rank	Junction	Readings	Mean ratio	Min. ratio	Mean delay %
1	Indiranagar	60	0.834	0.478	26.8
2	Hebbal	61	0.839	0.489	26.2
3	Bellandur	61	0.842	0.600	21.7
4	MG Road	60	0.855	0.500	21.6
5	Marathahalli	60	0.857	0.683	18.7
6	Whitefield	61	0.858	0.677	18.8
7	Mekhri Circle	57	0.868	0.500	19.4
8	Koramangala	60	0.869	0.424	20.0
9	Malleshwaram	58	0.879	0.500	17.9
10	Kengeri	57	0.882	0.625	16.2
11	Hennur	57	0.890	0.697	14.5
12	Domlur	58	0.894	0.697	13.8
13	Yeshwanthpur	60	0.900	0.682	13.6
14	HSR Layout	58	0.906	0.586	13.1
15	Majestic	61	0.916	0.625	11.8
16	Rajajinagar	58	0.932	0.706	8.6
17	Richmond Circle	58	0.934	0.643	8.8
18	Silk Board	60	0.936	0.575	10.0
19	Banashankari	58	0.938	0.784	7.3
20	KR Puram	61	0.939	0.667	8.2
21	Nagawara	58	0.944	0.760	6.6
22	Jayanagar	57	0.956	0.679	5.8
23	BTM Layout	58	0.978	0.750	2.7
24	Electronic City	61	0.994	0.885	0.7

iv. Analysis results

The hourly results are given in Table 11. The congestion is highest at 19:00, where the mean ratio is 0.723 and the mean speed is 22.8 km/h. It then decreases gradually: 0.776 at 20:00, 0.835 at 21:00, 0.923 at 22:00 and 0.975 at 23:00. Between midnight and 03:00 the mean ratio is between 0.974 and 0.998, which means that the roads are almost fully free flowing. In the morning, the value is 0.965 at 08:00, and drops to 0.837 at 09:00, which shows the start of the morning rush. The midday reading at 12:00 is also low (0.816).

Table 11: Hour-wise congestion results

Hour	Readings	Mean congestion ratio	Mean speed (km/h)
0:00	48	0.974	30.9
1:00	192	0.989	31.3
2:00	192	0.998	31.6
3:00	72	0.993	31.5
8:00	48	0.965	30.6
9:00	48	0.837	26.8
12:00	88	0.816	26.3
19:00	84	0.723	22.8
20:00	192	0.776	24.6
21:00	168	0.835	26.5
22:00	168	0.923	29.2
23:00	118	0.975	30.9

The combinations of junction and hour with the lowest mean ratio are listed in Table 12. All four of the worst combinations occur at 19:00 on Thursday, and they are Koramangala, Hebbal, Indiranagar and MG Road, with ratios between 0.50 and 0.54. This means that the vehicles on these segments were moving at only about half of the free-flow speed during the early evening.

Table 12: Junction and hour combinations with the lowest mean congestion ratio

Rank	Junction	Hour	Mean ratio
1	Koramangala	19:00	0.503
2	Hebbal	19:00	0.506
3	Indiranagar	19:00	0.531
4	MG Road	19:00	0.543
5	Indiranagar	12:00	0.583
6	Malleshwaram	12:00	0.599
7	Hebbal	20:00	0.601

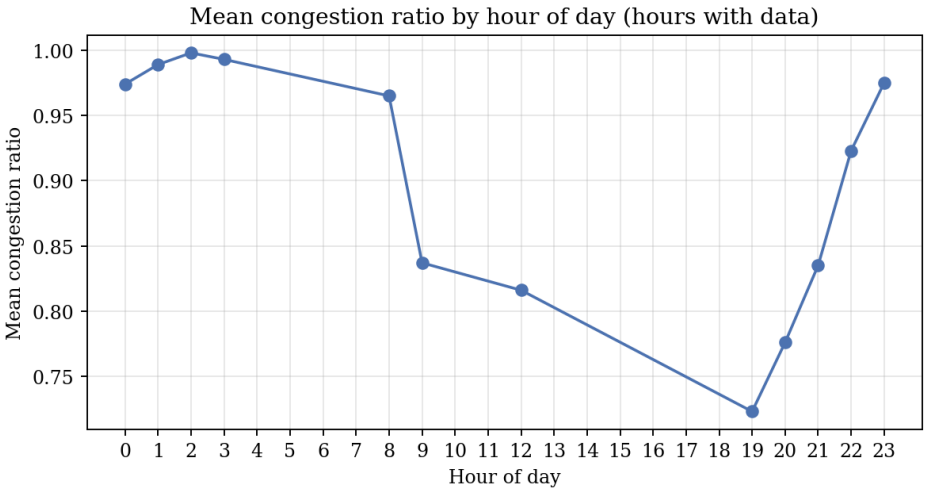


Figure 10: Mean congestion ratio by hour of the day

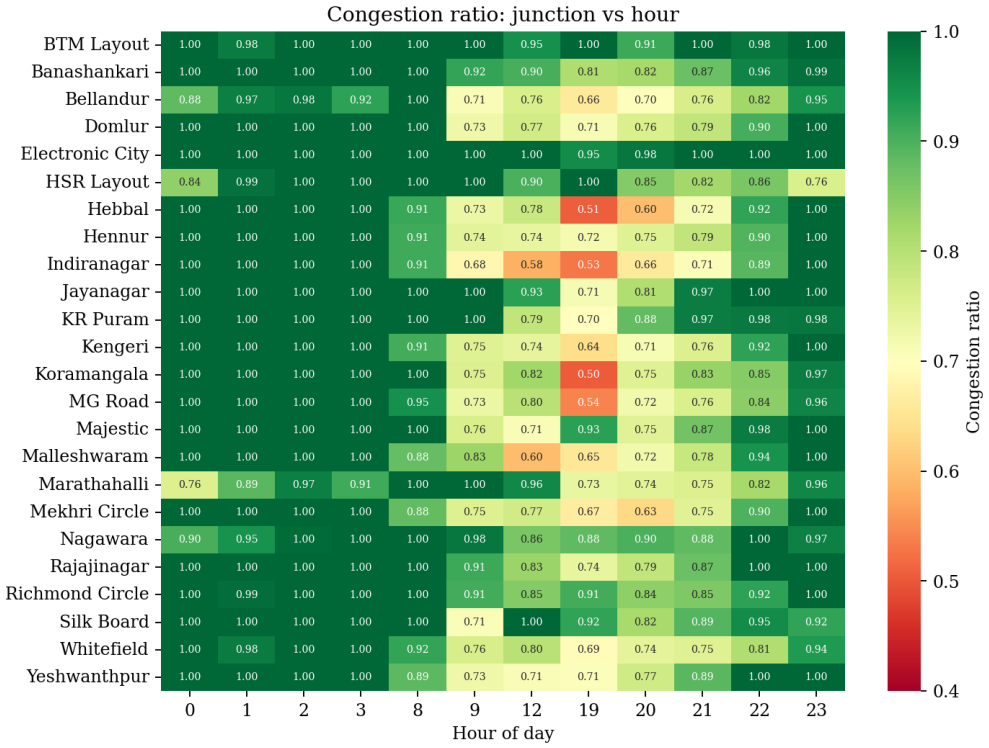


Figure 11: Heat map of the mean congestion ratio for each junction and hour

4.3 Analysis 3: Regression models (simple, multiple and polynomial)

i. Introduction

Regression is a supervised learning technique in which a model learns the relation between input variables and a continuous target. In this project the target is the congestion ratio of a junction. This analysis studies the regression family that is named in the syllabus: simple linear regression, multiple linear regression and polynomial regression, all estimated with the Ordinary Least Squares (OLS) method. The aim is to find out how much of the variation of

RMSE = square root of MSE *R-squared = 1 - SS_residual / SS_total*

MAE, MSE and RMSE are better when they are smaller, and R-squared is better when it is closer to 1. The settings used for all models in Chapter 4 are given in Table 14.

Table 14: Model and evaluation settings used in the analyses

Model	Setting
Baseline	Mean congestion ratio of the junction in the training data
Simple / Multiple Linear Regression	Ordinary least squares with 1 input (hour_cos) or 7 inputs
Polynomial Regression	Degree 2 or 3 terms of the 7 inputs, standardised, then least squares
Logistic Regression	Multinomial or binary, standardised inputs, class-balanced for 3 classes
Decision Tree	Maximum depth 6 (classifier) or 8 (regressor), at least 3 samples per leaf
Support Vector Machine / Regression	RBF kernel, C = 10, standardised inputs
Random Forest	300 trees, at least 2 samples per leaf, random_state = 42
Gradient Boosting	200 to 300 trees, learning rate 0.05, depth 3
MLP (neural network)	Two hidden layers (64 and 32 neurons), ReLU, Adam optimiser, standardised inputs
Train-test split	80 per cent training and 20 per cent test, random split with random_state = 42
Cross-validation	5 folds with shuffling, metric R-squared

iv. Analysis results

The simple linear regression between the free-flow speed and the current speed gave the line: current speed = 0.934 x free-flow speed - 0.99, with an R-squared of 0.742. The slope below 1 shows that, on average, the observed speed is lower than the free-flow speed, and the value of R-squared shows that the free-flow speed alone explains about 74 per cent of the variation of the current speed. The remaining variation is the effect of traffic.

The coefficients of the multiple linear regression estimated with OLS are given in Table 15. The two hour features are the strongest predictors, with t values of 20.5 and 18.2. The latitude (t = -6.7), the day of the week (t = -3.5), the weekend flag, and the longitude are also significant at the 5 per cent level, whereas the free-flow speed is not (p = 0.277). The weekday and the weekend flag have almost equal coefficients with opposite signs, which is a sign of multicollinearity, because the two variables carry nearly the same information in this short dataset. Their

77.8% accuracy, and the Logistic Regression is the weakest model with an accuracy of 68.7% and a macro F1-score of 0.527. The reason is that its decision boundary is linear in the inputs, whereas the dependence of congestion on the hour is cyclic and strongly non-linear.

Table 17: Comparison of the classifiers for the three congestion levels (test set)

Model	Accuracy	Macro precision	Macro recall	Macro F1	Log loss	AUC
Logistic Regression	68.7%	0.531	0.617	0.527	0.672	0.862
Decision Tree	77.8%	0.638	0.781	0.661	0.539	0.918
Support Vector Machine (RBF)	77.5%	0.633	0.745	0.662	0.386	0.937
Random Forest	85.2%	0.724	0.790	0.750	0.339	0.954
Gradient Boosting	85.9%	0.732	0.746	0.739	0.345	0.948
MLP (neural network)	84.5%	0.742	0.756	0.749	0.353	0.951

One advantage of a Decision Tree is that it can be written as rules. A tree of depth 3 was trained without class weights so that the rules are easy to read, and it reached an accuracy of 79.6% on the test set. Its eight rules are given in Table 18. The rules show that the hour of the day decides the first splits. The readings of the midday and evening hours are classified as Medium for the central and northern junctions and as Low for the southern junctions, the morning rush hour around 09:00 is classified as Medium, and the night hours are classified as Low. The High class does not appear in the rules, because it contains only about 5 per cent of the readings and a shallow tree has no leaf in which High is the majority. This is the reason why the balanced class weights and the deeper trees were used in the main comparison.

Table 18: Decision rules extracted from the depth-3 decision tree

No.	Rule (inputs are the encoded features)	Class	Readings
1	hour_cos <= 0.79 and hour_sin <= 0.35 and lat <= 12.93	Low	128
2	hour_cos <= 0.79 and hour_sin <= 0.35 and lat > 12.93	Medium	300
3	hour_cos <= 0.79 and hour_sin > 0.35 and hour_cos <= -0.60	Medium	40
4	hour_cos <= 0.79 and hour_sin > 0.35 and hour_cos > -0.60	Low	95
5	hour_cos > 0.79 and hour_sin <= -0.38 and weekday <= 3.50	Low	77
6	hour_cos > 0.79 and hour_sin <= -0.38 and weekday > 3.50	Low	56
7	hour_cos > 0.79 and hour_sin > -0.38 and lon <= 77.64	Low	322
8	hour_cos > 0.79 and hour_sin > -0.38 and lon > 77.64	Low	116

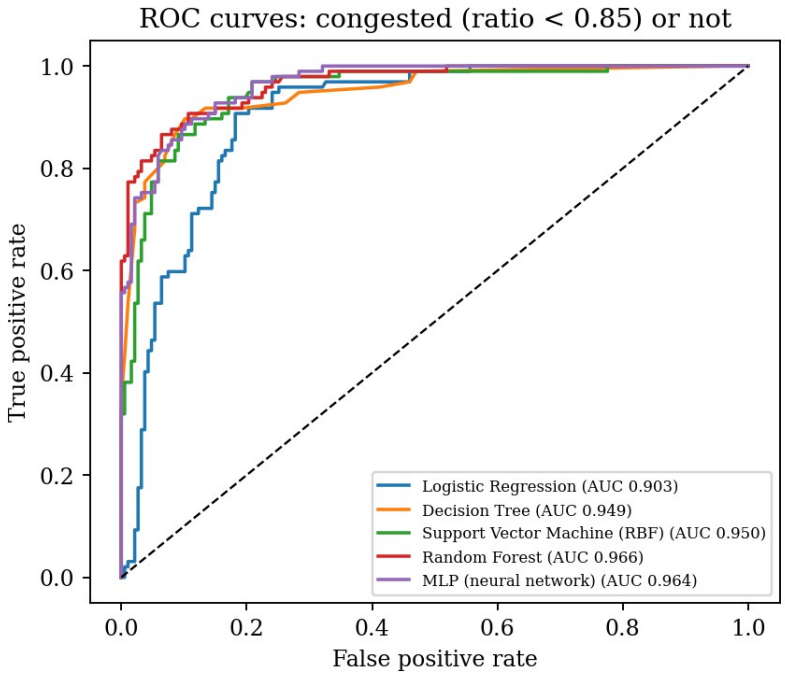


Figure 14: ROC curves of the classifiers for congested versus not congested

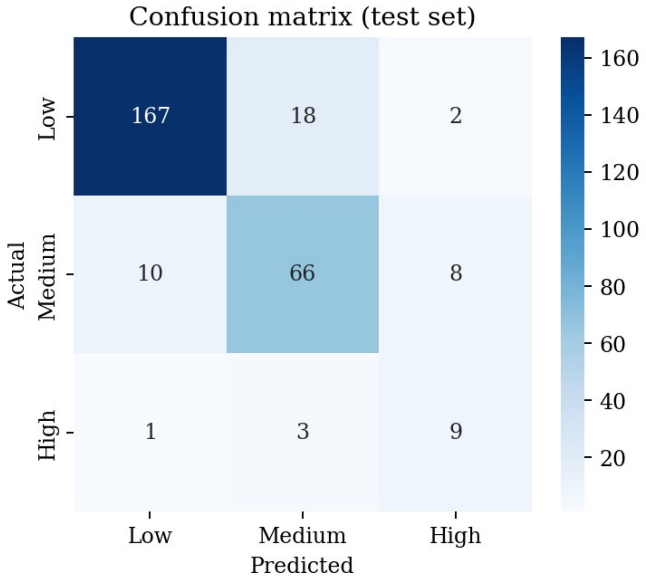


Figure 15: Confusion matrix of the Random Forest classifier

4.5 Analysis 5: Clustering of junctions

i. Introduction

Clustering is an unsupervised learning technique, which means that the data has no target label and the algorithm looks for groups of similar objects on its own. Here the objects are the 24 junctions. The question is whether the junctions can be divided into groups with a similar congestion behaviour, for example a group of junctions that are heavily loaded in the evening

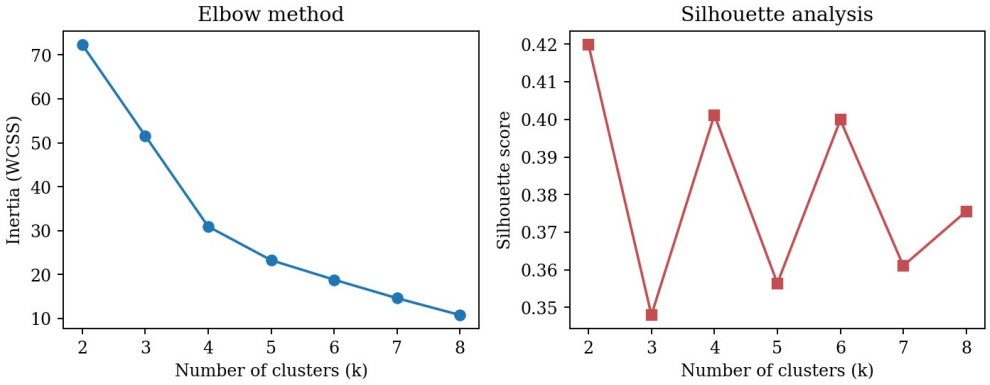


Figure 16: Elbow method and silhouette analysis for K-Means

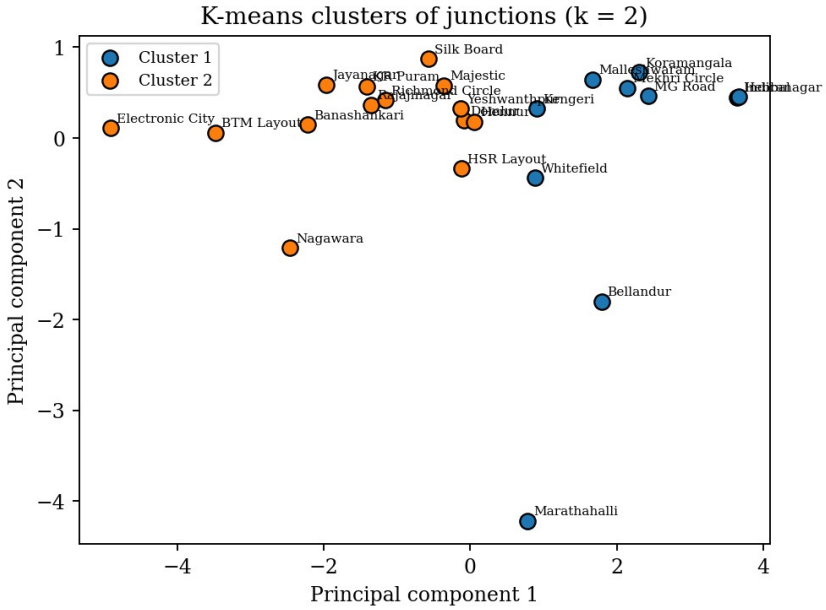


Figure 17: K-Means clusters of the junctions

If these junctions are congested	then these are also congested	Support	Confidence	Lift
Richmond Circle	Indiranagar, Majestic	0.204	0.846	2.86
Indiranagar, Richmond Circle	Majestic	0.204	0.846	2.86
Richmond Circle	Kengeri, Majestic	0.204	0.846	2.86

These rules must be interpreted with care. Only 54 rounds are available, and each rule is supported by only 11 to 14 rounds. Congestion is also strongly linked to the time of the day, so many junctions become congested together simply because the round was taken in the evening peak. The rules therefore describe co-occurrence and not cause and effect. They do, however, show how the method works and which neighbouring junctions would be interesting to study with a longer dataset.

v. Visualization

The result of this analysis is a set of rules, and it is presented in Table 26 instead of a chart. The support of the single junctions mentioned above shows that congestion was a common event: in about half of the rounds, Bellandur, Marathahalli and Whitefield were already below the congestion limit of 0.85.

4.7 Analysis 7: Dimensionality reduction with PCA

i. Introduction

Principal Component Analysis (PCA) is an unsupervised technique for dimensionality reduction. It replaces a set of correlated variables with a smaller number of new variables, called principal components, that are linear combinations of the original variables and that are uncorrelated with each other. The first component captures the largest possible share of the variance, the second component captures the largest share of the remaining variance, and so on. PCA is useful for visualising high-dimensional data and for removing redundancy.

ii. General description

PCA was applied to eight numeric attributes of every reading: the current speed, the free-flow speed, the congestion ratio, the delay percentage, the sine and cosine of the hour, the latitude and the longitude. The variables were standardised before the analysis, because PCA is sensitive to the scale of the variables. In a second experiment, called principal component

component is dominated by the cosine (0.76) and the sine (-0.56) of the hour, so it is a time-of-day axis.

Table 28: Loadings of the first three principal components

Variable	PC1	PC2	PC3
current_speed	0.457	0.486	-0.033
free_flow_speed	0.223	0.721	-0.055
congestion_ratio	0.538	-0.268	0.038
delay_pct	-0.530	0.270	-0.036
hour_sin	0.341	-0.231	-0.558
hour_cos	0.204	-0.137	0.765
lat	-0.113	-0.117	-0.227
lon	-0.017	0.114	0.214

In the principal component regression of Table 29, the R-squared on the test set is only 0.208 with one component and 0.280 with five components, and it rises to 0.470 with six components and 0.491 with all seven. This is an important lesson: PCA orders the components by the variance in the inputs and not by their usefulness for the target. In this dataset the components that are most useful for predicting congestion are the later ones, so reducing the dimensions with PCA does not help here, and the original features are kept for the models.

Table 29: Principal component regression on the seven model inputs (test set)

Components kept	MAE	RMSE	R-squared
1	0.0951	0.1184	0.208
2	0.0954	0.1184	0.208
3	0.0914	0.1138	0.268
4	0.0914	0.1139	0.266
5	0.0897	0.1129	0.280
6	0.0696	0.0968	0.470
7	0.0674	0.0949	0.491

v. Visualization

Figure 19 shows the scree plot, where the bars are the variance of each component and the line is the cumulative variance. Figure 20 projects all the readings on the first two components and colours them by the congestion level. The Low readings are mostly on the right side, where PC1

Because of the non-linear activation functions in the hidden layers, an MLP can learn complex non-linear relations, which makes it a natural candidate for the cyclic effect of the time of day on traffic.

ii. General description

An MLP with two hidden layers of 64 and 32 neurons was used for the regression of the congestion ratio and also as a classifier of the congestion level and of the congested or not problem. The seven inputs were standardised, because neural networks train poorly when the inputs have different scales. The network has 7 neurons in the input layer and 1 neuron in the output layer for regression, or 3 and 1 output neurons for the three-class and two-class classification.

iii. Specific requirements, functions and formulas

Each neuron computes a weighted sum of its inputs plus a bias and passes the sum through an activation function. The Rectified Linear Unit (ReLU) was used in the hidden layers. The weights are learned by backpropagation, which calculates the gradient of the loss with respect to every weight, and the Adam optimiser, which updates the weights in the direction that reduces the loss. The loss is the squared error for regression and the log loss for classification. The MLPRegressor and MLPClassifier classes of scikit-learn were used.

$$h = ReLU(W x + b) = max(0, W x + b) \qquad Loss\ (regression) = (1/n) \times Sum\ (y - y_hat)^2$$

iv. Analysis results

The training of the regression network converged in 43 iterations with a final training loss of 0.0037. The results are summarised in Table 30. As a regressor the MLP reaches an R-squared of 0.604 (cross-validated 0.530) with an RMSE of 0.084, which is better than the linear and polynomial models of degree 2 but lower than the tree ensembles. As a classifier it is competitive: its accuracy is 84.5%, its macro F1-score is 0.749 and its AUC is 0.951 for the three classes, and the AUC is 0.964 for the congested or not problem. The dataset has only 1134 training readings, which is small for a neural network, and this is the likely reason why the MLP does not exceed the Random Forest.

Table 30: Results of the MLP network

Task	Measure	Value
Regression of the congestion ratio	MAE	0.0571
	RMSE	0.0837

4.10 Overall discussion and reliability of the results

Taken together, the nine analyses give a consistent picture of the traffic observed in Bangalore during the data collection. The congestion is concentrated in the early evening, with the lowest speeds at 19:00, and gradually reduces during the night. A group of junctions, including Indiranagar, Hebbal, Bellandur, MG Road, Marathahalli, Koramangala and Whitefield, repeatedly shows lower speeds than the others. Time of the day is the most important factor for predicting congestion, followed by the location.

The predictive results are encouraging, but they must be interpreted with caution for four reasons. First, only about two and a half days of data were available. Second, only 12 of the 24 hours of the day are present in the data, so the model has never seen the afternoon and the morning rush hours between 04:00 and 07:00 or 13:00 and 18:00. Third, there are no weekend readings during the daytime, so the model cannot learn a weekend effect, and the feature importance of the weekday is therefore close to zero. Fourth, the random split has some leakage between neighbouring readings, which is why a second evaluation on completely unseen hours was added. For these reasons the models in this report should be regarded as a working prototype that shows the complete method, and not as a final traffic forecasting system. The same pipeline can be applied without changes to a longer dataset, and the accuracy can be expected to become more reliable as more days and hours are collected.

5. Conclusion

This project demonstrated the complete workflow of a predictive analytics project on real-time traffic data from Bangalore. A Python script collected 1418 readings from the TomTom Traffic Flow API for 24 locations between 29 and 31 October 2026. The data was cleaned and validated, and the congestion ratio, the delay and the congestion level were derived from it. The exploratory analysis showed that the average speed was about 10 per cent below the free-flow speed, that the early evening around 19:00 was the most congested time in the data, and that the roads were almost free flowing between midnight and 03:00.

The junction-wise analysis showed that Indiranagar, Hebbal, Bellandur, MG Road and Marathahalli had the lowest mean congestion ratios of all junctions, between 0.83 and 0.86, while Electronic City, BTM Layout and Jayanagar had the highest. The time-of-day analysis showed that Koramangala, Hebbal, Indiranagar and MG Road were the worst junctions at 19:00, with speeds of only about half of the free-flow speed. A small comparison of the same hours on Thursday and Friday suggested that Friday evening was slightly more congested.

In the predictive part, a Random Forest regressor estimated the congestion ratio with an RMSE of 0.060 and an R-squared of 0.796 on a random test set and 0.621 on hours that were not seen in training, and it clearly outperformed both a junction-average baseline and Linear Regression. A Random Forest classifier assigned the correct congestion level to 85.2% of the test readings, compared with 65.8% for a model that always predicts Low. The hour of the day was found to be the most important predictor, followed by the location of the junction.

The syllabus techniques gave the following additional results. Among the regression models, the polynomial regression of degree 3 improved the R-squared from 0.491 (multiple linear regression) to 0.691, and logistic regression separated congested from free-flowing readings with an AUC of 0.903. K-Means grouped the 24 junctions into a heavily loaded cluster of 10 junctions and a lighter cluster of 14 junctions, hierarchical clustering showed that the profiles form a continuous range, and association rules revealed groups of neighbouring junctions that become congested together. PCA showed that three components describing congestion, road speed class and time of day carry most of the information, the MLP neural network was competitive for classification, and the comparison of a single tree with bagging and the Random Forest showed the reduction of variance that ensemble learning gives.

The project also showed the practical difficulties of working with real data: gaps in the collection because the collecting computer was not always running, network and certificate

6. Future Scope

There are many ways in which this work can be extended.

- Longer and continuous data collection: running the collection for several weeks without gaps, and on a machine or cloud service that stays on, would give all 24 hours and all seven days. This would allow a reliable weekday and weekend comparison and the learning of weekly patterns.
- Time-aware validation: with a longer dataset, the models should be validated by training on earlier days and testing on later days, so that the result reflects real forecasting.
- Additional features: weather information such as rain, public holidays, festivals, school timings, road works and accidents strongly influence traffic and could be added from other APIs. The Traffic Incidents API of TomTom could provide accidents and closures directly.
- Better location representation: using several points per junction and the length and class of the road segment would reduce the problem of an unrepresentative segment, as seen for Silk Board.
- Advanced models: gradient boosting methods such as XGBoost and LightGBM, and time series or deep learning models such as LSTM networks, can be compared with the Random Forest used here.
- Forecasting ahead in time: instead of estimating the ratio at the same time, the models can be trained to forecast the congestion 15, 30 or 60 minutes ahead using recent readings as inputs.
- Interactive application: a dashboard, for example in Streamlit, could allow a user to choose a junction and a time and see the predicted congestion level and a map of the monitored locations.
- Route-level analysis: combining segment predictions along a route could estimate the travel time between two places and suggest the best departure time.

7. References

[1] TomTom Developers. Traffic Flow API and Flow Segment Data documentation. TomTom International BV. Available at https://developer.tomtom.com (accessed October 2026).

[2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

[3] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[4] Hastie, T., Tibshirani, R. and Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction (2nd ed.). Springer.

[5] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 56-61.

[6] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. Nature, 585, 357-362.

[7] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science and Engineering, 9(3), 90-95.

[8] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

[9] Breiman, L. (1996). Bagging predictors. Machine Learning, 24(2), 123-140.

[10] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.

[11] Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. Journal of Computer and System Sciences, 55(1), 119-139.

[12] Cortes, C. and Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297.

[13] Arthur, D. and Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. Proceedings of the 18th ACM-SIAM Symposium on Discrete Algorithms, 1027-1035.

[14] Agrawal, R. and Srikant, R. (1994). Fast algorithms for mining association rules. Proceedings of the 20th International Conference on Very Large Data Bases, 487-499.

[15] Jolliffe, I. T. (2002). Principal Component Analysis (2nd ed.). Springer.

[16] Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations.

[17] Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods, 17, 261-272.

[18] Python Software Foundation. Python Language Reference. Available at https://www.python.org (accessed October 2026).